BIT 326 DISTRIBUTED SYSTEMS

Chapter 10 - Distributed systems security

10.1. Introduction

A distributed system is typically a composition of geo-dispersed resources (computing and communication) that collectively

1) provides services that link dispersed data producers and consumers,
2) provides on-demand, highly reliable, highly available, and consistent resource access, often using replication schemas to handle resource failures, and
3) enables a collective aggregated capability (computational or services) from the distributed resources to provide (an illusion of) a logically centralised/coordinated resource or service.

The distributed resources are typically dispersed to provide various features to the users. These include; geo-proximate and low-latency access to computing elements, high bandwidth and high-performance resource access, and especially highly-available uninterrupted services in the case of resource failure or deliberate breaches. The overall technical needs in a distributed system consequently relate to the orchestration of the distributed resources such that the user can transparently access the enhanced services arising from the distribution of resources without having to deal with the technical mechanisms providing the varied forms of distributed resource and service orchestrations

To support these functionalities, a distributed system commonly entails a progression of four elements. These include

1) data flows across the collection of authorised inputs (regulated via Access/Admission Control),
2) transportation of the data to/across the distributed resources (Data Transport functionality),
3) a resource coordination schema (Coordination Services),
4) property based (e.g., time or event-based ordering, consensus, virtualisation) data management to support the desired applications such as transactions, databases, storage, control, and computing.

10.2. Security Concerns in Distributed Systems

Security concerns in distributed systems arise from decentralization, expanded attack surfaces across multiple nodes, and communication over untrusted networks. The concerns includes;

1) Data Confidentiality: ensuring that sensitive, private, or proprietary information is accessed, used, or disclosed only by authorized individuals, protecting it from theft, breaches, or unauthorized viewing
2) Data integrity: ensuring the accuracy, consistency, and reliability of data throughout its lifecycle, from creation to deletion. It maintains data quality and trustworthiness, preventing corruption from human error, unauthorized access, or technical failures
3) Access Control: regulating who or what can view or use resources in a computing environment or physical space. It limits access to sensitive data and locations by identifying, authenticating, and authorizing users
4) Secure Communication: protecting data-voice, video, and text from interception and unauthorized access, ensuring confidentiality, integrity, and authentication.

5) Availability and Resilience: ensuring systems remain operational and recover swiftly from disruptions.
   - Availability (Pre-emptive): Focuses on minimizing downtime through redundancy, load balancing, and failover mechanisms to eliminate single points of failure.
   - Resilience (Reactive/Adaptive): Encompasses high availability, fault tolerance, and disaster recovery (DR). It ensures service continuity even during disasters.
6) Denial of Service (DoS): Attackers overwhelm the bandwidth or resources of multiple nodes, disrupting service availability.
7) Non-repudiation: ensures that parties involved in a transaction or communication cannot deny their actions, such as sending a message, signing a contract, or authorizing a transaction, after the fact.
8) Data Consistency: Maintaining data consistency across asynchronous nodes is difficult,
9) Decentralized Control: No single administrator overseeing the entire system makes maintaining consistent security standards and patching difficult.

10.3. Goals and Objectives of Distributed System Security

Distributed Systems security protects data, services, and communications across interconnected nodes from unauthorized access, tampering, and disruption. Key goals of distributed system security include:

1) **Confidentiality:** Protecting sensitive information from unauthorized access during storage, processing, or transmission, often through encryption.
2) **Integrity:** Ensuring that data remains accurate and unaltered, guarding against tampering with information, messages, or code.
3) **Availability:** Ensuring that resources, data, and services are accessible to authorized users when needed, protecting against denial-of-service (DoS) attacks.
4) **Authentication:** Verifying the identity of users, processes, and systems to prevent unauthorized access.
5) **Authorization (Access Control):** Controlling access to resources, ensuring that users can only access the data and services they are permitted to use.
6) **Non-Repudiation:** Preventing participants in a transaction from denying their actions, such as proving that a message was sent or received.
7) **Resilience and Fault Tolerance:** Designing the system to withstand attacks and component failures without shutting down, maintaining security even when parts of the system are compromised.
8) **Auditability/Accountability:** Tracking and logging activities within the distributed system to detect, investigate, and hold users responsible for actions.
9) **Secure Communication:** Using protocols like TLS/SSL to protect data-in-transit between nodes from eavesdropping and tampering. SSL (Secure Sockets Layer) and its modern successor, TLS (Transport Layer Security), are cryptographic protocols that encrypt data sent between a browser and a server, ensuring privacy and authentication

10.4. Security Mechanisms in Distributed Systems

Security mechanisms in distributed systems protect data confidentiality, integrity, and availability across decentralized components. Key Mechanisms & Techniques includes;

1) **Authentication Mechanisms in Distributed System:** ensure that users and services are who they claim to be before granting access to resources. Here are the key authentication mechanisms:
   a) **Password-based Authentication:** Users authenticate with a username and password. Commonly used but vulnerable to password breaches and phishing attacks.
   b) **Multi-factor Authentication (MFA):** Requires users to provide two or more authentication factors (e.g., password + OTP, fingerprint). Enhances security by adding an extra layer of verification.
   c) **Token-based Authentication:** Uses tokens (e.g., JWT, OAuth tokens) for authentication. Tokens are generated by an authentication server and validated by services.
   d) **Biometric Authentication:** Uses unique biological traits (e.g., fingerprints, facial recognition) for authentication. Provides strong authentication but may require specialized hardware.
   e) **Certificate-based Authentication:** Uses digital certificates to authenticate clients and servers. Certificates are issued by trusted Certificate Authorities (CAs) and verify identity.
   f) **Single Sign-On (SSO):** Allows users to authenticate once and gain access to multiple systems. Improves user experience and reduces password fatigue.
2) **Authorization Controls in Distributed System:** Dictate what actions users and services are permitted to perform on resources. Here are the key authorization controls:
   a) **Role-Based Access Control (RBAC):** Assigns permissions based on roles (e.g., admin, user, manager). Simplifies access management by grouping users with similar responsibilities.
   b) **Attribute-Based Access Control (ABAC):** Grants access based on attributes (e.g., user properties, resource properties, environmental conditions). Provides fine-grained access control tailored to specific contexts.
   c) **Access Control Lists (ACLs):** Lists of permissions associated with users or groups. Applied directly to resources to specify who can access them and what actions they can perform.
   d) **Policy-Based Access Control:** Uses policies to define access rules based on conditions and attributes. Offers flexibility to adapt access controls dynamically based on changing conditions.
   e) **Mandatory Access Control (MAC):** Assigns labels (e.g., sensitivity levels) to resources and subjects. Access decisions are based on predefined security policies set by administrators.
   f) **Role-Based Access Control (RBAC):** Assigns permissions based on roles (e.g., admin, user, manager). Simplifies access management by grouping users with similar responsibilities.
3) **Data Protection and Encryption in Distributed System (Cryptographic Protection):**
   Data is encrypted at rest and in transit using symmetric (AES) and asymmetric (RSA) algorithms. End-to-end encryption ensures only communicating parties can decrypt the data. Data protection and encryption in distributed systems are essential for ensuring confidentiality, integrity, and availability of sensitive information across interconnected networks.
   a) **AES (Advanced Encryption Standard) and RSA (Rivest-Shamir-Adleman) Crytographic algorithms:**
      - Converts plaintext data into ciphertext.
      - Ensures data confidentiality during transmission and storage.
   b) **Transport Layer Security (TLS) / Secure Socket Layer (SSL):**
      - Protocols that encrypt data exchanged between clients and servers.
      - Establish secure communication channels to prevent eavesdropping and tampering.

c) **End-to-End Encryption (E2EE):**
   - Encrypts data at the source and decrypts it only at the destination.
   - Protects data from interception or surveillance during transmission.

d) **Data Masking and Tokenization:**
   - Techniques to obscure sensitive data (e.g., masking credit card numbers, tokenizing data).
   - Reduces exposure of sensitive information in non-production environments.

e) **Key Management:**
   - Manages encryption keys used to encrypt and decrypt data.
   - Ensures secure storage, rotation, and distribution of keys to authorized entities.

f) **Digital Signature**:
   - by using asymmetric cryptography (private/public key pairs) to verify messages across decentralized nodes
   - provide essential security—authenticity, integrity, and non-repudiation

4) **Secure Communication in Distributed System:** Secure communication in distributed systems is essential for maintaining the confidentiality, integrity, and authenticity of data exchanged between various components across a network. It ensures that data is protected from eavesdropping, tampering, and impersonation, thereby preserving the privacy and security of sensitive information.

a) **Encryption Protocols:**

Transport Layer Security (TLS)/Secure Socket Layer (SSL): TLS and SSL are cryptographic protocols designed to provide secure communication over a computer network. This encryption is achieved using a combination of symmetric and asymmetric encryption, with certificates issued by trusted Certificate Authorities (CAs) to authenticate the server's identity.

b) **End-to-End Encryption (E2EE):**

End-to-end encryption ensures that data is encrypted on the sender's side and only decrypted on the receiver's side, preventing intermediaries from accessing the plaintext data. This method is crucial for protecting data in transit, especially in messaging applications and VoIP services, where data security is paramount.

c) **Secure Channels and Protocols:**

Internet Protocol Security (IPsec): IPsec is a protocol suite that provides secure IP communications by authenticating and encrypting each IP packet of a communication session. It operates at the network layer, ensuring that all traffic between two endpoints is secure, regardless of the application or protocol being used.

d) **Authentication Mechanisms:**
   - **Public Key Infrastructure (PKI):** PKI supports secure communication by providing a framework for managing digital certificates and public-key encryption. It ensures that the parties involved in a communication session are authenticated and that the integrity of the data is maintained.
   - **HMAC (Hash-Based Message Authentication Code):** HMAC provides a way to verify the integrity and authenticity of a message. It combines a cryptographic hash function with a secret key, ensuring that the message has not been altered in transit and confirming the identity of the sender.

     Hash functions have three critical properties:

i. Any change in data produces a different hash
ii. The same data always produces the same hash
iii. It is infeasible to reconstruct data from its hash

e) **Transport Layer Security Enhancements:**
   - **Perfect Forward Secrecy (PFS):** PFS ensures that session keys are not compromised even if the server's long-term keys are exposed. By generating unique session keys for each communication session, PFS provides an additional layer of security.
   - **Certificate Pinning:** Certificate pinning involves associating a host with a specific certificate or public key. This practice helps protect against man-in-the-middle (MITM) attacks by ensuring that the client connects only to the server with the expected certificate.

5) **Mutual TLS (mTLS):** In a microservices architecture, mTLS ensures that both the client and server verify each other's certificates, preventing unauthorized services from interacting with the system.
6) **Data Integrity Check:** Hash functions and Message Authentication Codes (MAC) detect whether data has been modified during transmission.
7) **Key Management:** Secures the lifecycle of encryption keys (generation, storage, rotation, and distribution) to protect data.
8) **Auditing and Monitoring:** Log management and audit logs allow for auditing actions performed within the system, crucial for detecting breaches and ensuring accountability.
9) **Security Policy and Management:** Defines the overall strategy, including auditing (logging activities to detect breaches), and securing communications between clients and servers.
10) **Threat Prevention & Detection:** Techniques to protect against malicious activities, such as Denial of Service (DoS) mitigation, system monitoring, and Intrusion Detection Systems (IDS).

Advanced Security Architectures

11) **Zero Trust Architecture (ZTA):** Adopts a "never trust, always verify" approach, assuming breaches can occur and requiring continuous verification of every user and device, regardless of location.
12) **Proxy-based Security:** An API gateway or ingress proxy (e.g., in Kubernetes) acts as a central entry point, handling authentication and terminating SSL, shielding internal services.
13) **Blockchain-based Security:** Uses immutable ledgers to manage decentralized identities (DIDs) and secure consensus mechanisms, protecting against data tampering

10.5. Challenges in Distributed System Security

Security in a distributed system poses unique challenges that need to be considered when designing and implementing systems. A compromised computer or network may not be the only location where data is at risk; other systems or segments may also become infected with malicious code. In the past, security was typically handled on an end-to-end basis. All the work involved in ensuring safety occurred "within" a single system and was controlled by one or two administrators. The rise of distributed systems has created a new ecosystem that brings with it unique challenges to security. Distributed systems are made up of multiple nodes working together to achieve a common goal, these nodes are usually called peers.

Securing distributed systems poses several significant challenges due to their complexity, scale, and dynamic nature. Here are the key challenges in distributed system security:

1) **Network Complexity:** Increases the attack surface and complexity of managing security configurations and updates across diverse network environments.
2) **Data Protection and Encryption:** Vulnerabilities in encryption implementations or weak key management practices can lead to data breaches and unauthorized access.
3) **Authentication and Authorization:** Misconfigurations or vulnerabilities in authentication mechanisms can lead to unauthorized access and compromise the integrity of the entire system.
4) **Diverse Technologies and Platforms:** Compatibility issues, differing security postures, and varying levels of support for security standards can introduce vulnerabilities and complexities in maintaining a consistent security posture.
5) **Scalability and Performance:** Security measures such as encryption and authentication may introduce latency and overhead, affecting system performance and responsiveness, especially under high load conditions
6) **Increased Attack Surface:** Multiple nodes and communication channels provide more entry points for attackers.
7) **Distributed Trust: Security mechanisms** must be decentralized, as there is no single central point of control, though certificate authorities might be used.
8) **Fault Tolerance:** Ensuring the system remains secure and functional even if some components are compromised or fail.
9) **Consistency:** Maintaining data integrity across nodes using consensus protocols.